
Machine Learning 3

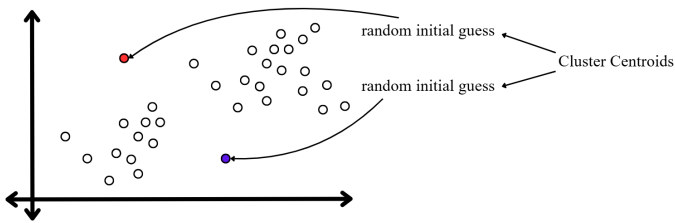
This section will cover algorithms that deal with Unsupervised Learning

Clustering

K-means Intuition

- Assign each point to its closest relative
- Recompute the centroids:
 - find average location of the labelled points from one category, wherever that is move the centroid there (loop this process).

Example:



K-Means Algorithm

Algorithm 1 K-Means Clustering

```
1: Randomly initialise  $K$  cluster centroids  $\mu_1, \mu_2, \dots, \mu_K \in \mathbb{R}^n$ 
2: repeat
3:   for  $i = 1$  to  $m$  do
4:      $c^{(i)} \leftarrow \arg \min_k \|x^{(i)} - \mu_k\|^2$ 
5:   end for
6:   for  $k = 1$  to  $K$  do
7:      $\mu_k \leftarrow \frac{1}{|C_k|} \sum_{i: c^{(i)}=k} x^{(i)}$ 
8:   end for
9: until convergence
```

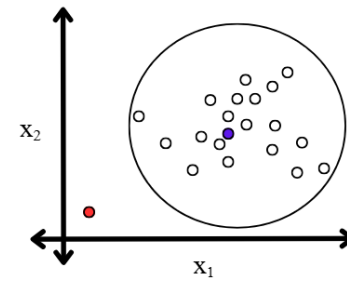
Note: $c^{(i)}$ is the index (from 1 to K) of the cluster centroid closest to $x^{(i)}$, i.e., the assignment step finds the nearest centroid by squared Euclidean distance. The update step then moves each centroid μ_k to the mean of all points currently assigned to cluster k .

Example:

Say 4 points:

$$\mu_1 = \frac{1}{4} [x^{(1)} + x^{(5)} + x^{(6)} + x^{(10)}]$$

Note: Sometimes we end up getting 0 points in a cluster because of problematic initial positions.

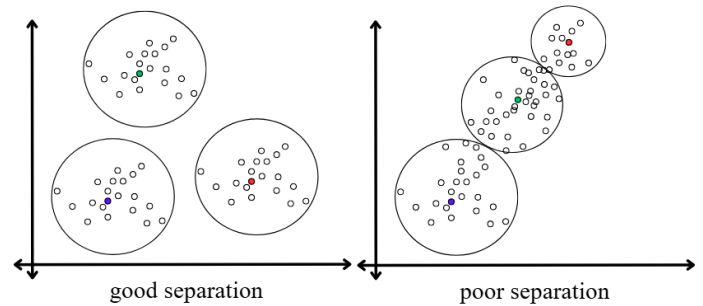


Solution:

- Eliminate the cluster with 0 points we get (K-1) clusters instead of K.
- Re-initialise random-centroids of k-clusters

Example:

K-Means clustering is also applied to not well separated data



Note: The clustering of not well-separated data might be useful in applications where you wish to force delineations, such as T-shirt sizes: you only want 3 different ones but need to figure out the measurements for each.

K-Means Optimisation Objective

- $c^{(i)}$ = index of cluster (1, 2, ..., K) to which example $x^{(i)}$ is currently assigned
- μ_k = cluster centroid k
- $\mu_k^{(i)}$ = cluster centroid of cluster to which example $x^{(i)}$ has been assigned.

Cost Function

$$J(c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K) = \frac{1}{m} \sum_{i=1}^m \underbrace{\|x^{(i)} - \mu_{c^{(i)}}\|^2}_{\text{Distortion Cost Function}}$$

Our objective is to:

$$\min_{c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K} J(c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K)$$

Random initialisation

- choose $K \ll m$
- Randomly pick K training examples
- set $\mu_1, \dots, \mu_K$ equal to those K examples

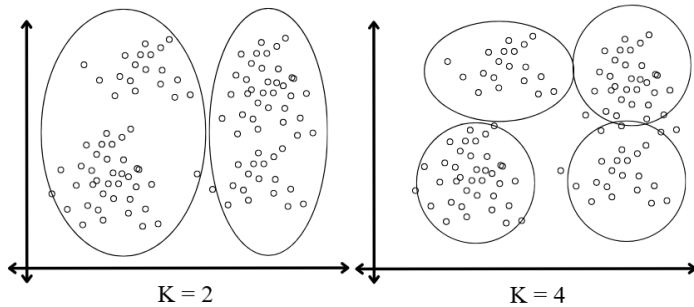
Note: Since, different centroid initialisations will give different clustering(s) we want to sample a wide variety of them (compared with their corresponding cost functions).

Algorithm 2 K-Means with Random Restarts

```
1: for  $i = 1$  to 100 do ▷ typically 50–1000
2:   Randomly initialise  $K$  centroids by sampling  $K$  distinct
   training examples
3:   Run K-Means to obtain  $c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K$ 
4:   Compute distortion  $J^{(i)} \leftarrow \frac{1}{m} \sum_{i=1}^m \|x^{(i)} - \mu_{c^{(i)}}\|^2$ 
5: end for
6: return  $\{c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K\}$  corresponding to
    $\min_i J^{(i)}$ 
```

Example:

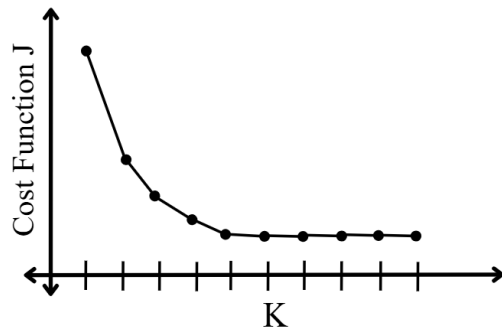
There can be 2 or more right answers for differing choices of K .



Choosing the value of K

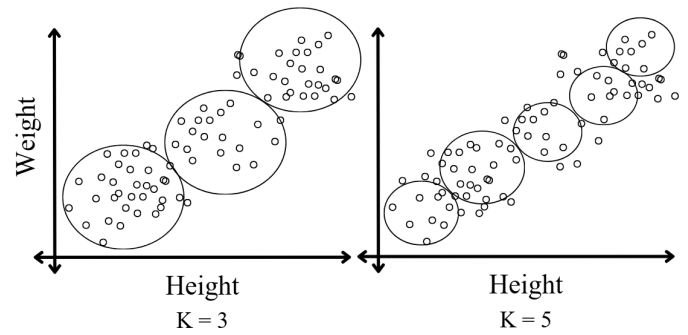
Note: The "right" value of K is often ambiguous and there is no one right answer, also it might be counterproductive to our application to minimise the cost function, J .

Elbow Method



Note: Often, you want to get clusters for some later (down-stream) purpose. Evaluate K-Means based on how-well it performs on that later purpose not just with the value of K that occurs at the low-sharp bend of the J v/s K graph.

Example: 2 companies want 3 v/s 5 different shirt sizes



Note: First company has shirt sizes as S, M, L whereas the second has XS, X, M, L, XL.

Anomaly-Detection

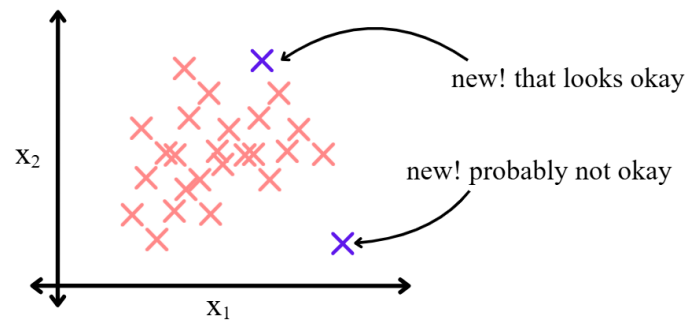
Example: Aircraft engine features:

- x_1 = heat generated
- x_2 = vibrational intensity
- $\vdots$

With this we could have a dataset,

$$\text{Dataset: } \{x^{(1)}, x^{(2)}, \dots, x^{(m)}\}$$

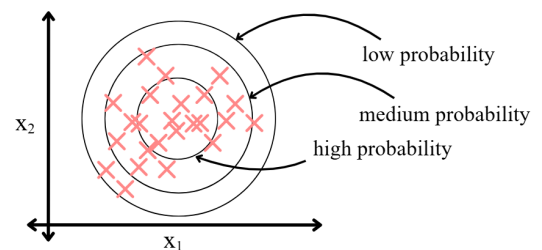
If we then have a new engine: x_{test} , we could be interested in knowing if its behaviour is anomalous compared to good examples in our dataset.



Density Estimation

$$\begin{cases} \text{if } P(X_{test}) < \epsilon \rightarrow \text{unlikely; anomalous} \\ \text{if } P(X_{test}) \geq \epsilon \rightarrow \text{likely; non-anomalous} \end{cases}$$

Example:



Examples: Applications

– Fraud Detection:

- $x^{(i)}$ = features of user i's activities
 - typing speed,
 - frequency of login,
 - etc, ...

– Manufacturing:

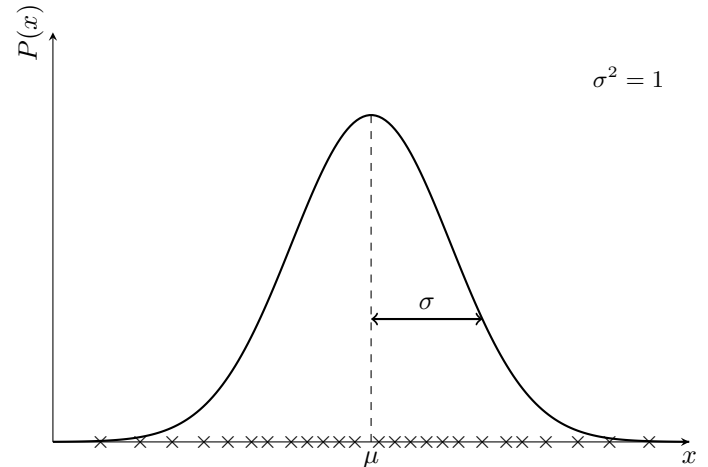
- $x^{(i)}$ = features of product i

– Monitoring computers in Data-centers:

- $x^{(i)}$ = features of machine i
 - x_1 = memory use,
 - x_2 = number of disk access(es)/second,
 - x_3 = CPU Load,
 - $\vdots$

Example: Density Estimation with Gaussian Distribution

$$\text{Dataset: } = \{x^{(1)}, x^{(2)}, \dots, x^{(m)}\}$$

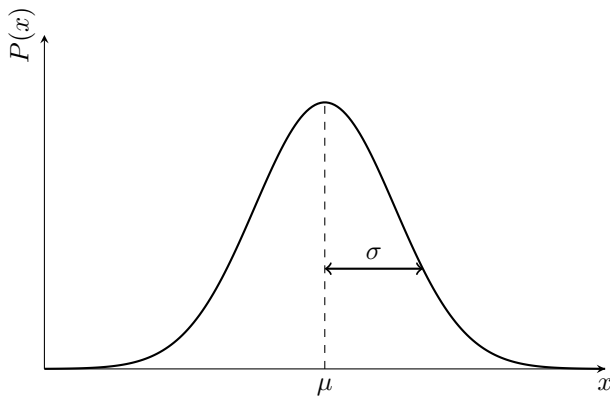


Gaussian (Normal) Distribution

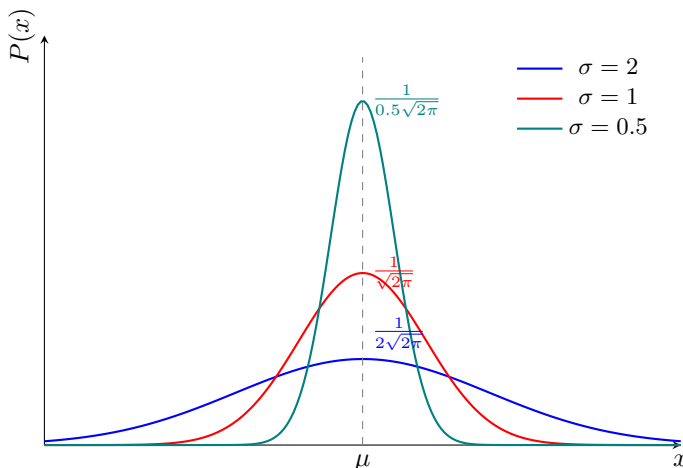
Say x is a number.

Probability of x , $P(x)$ is determined by a Gaussian with mean μ , variance σ^2 (σ is standard deviation).

$$P(x) = \frac{1}{\sqrt{2\pi} \cdot \sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$



Note: The area under the curve needs to be 1, so as σ drops the curve becomes skinnier but taller as well,



$$\mu = \frac{1}{m} \sum_{i=1}^m x^{(i)} - (*)$$

$$\sigma^2 = \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \mu)^2 - (**)$$

Note: (*), (**) are called the **maximum likelihood estimations** for μ and σ

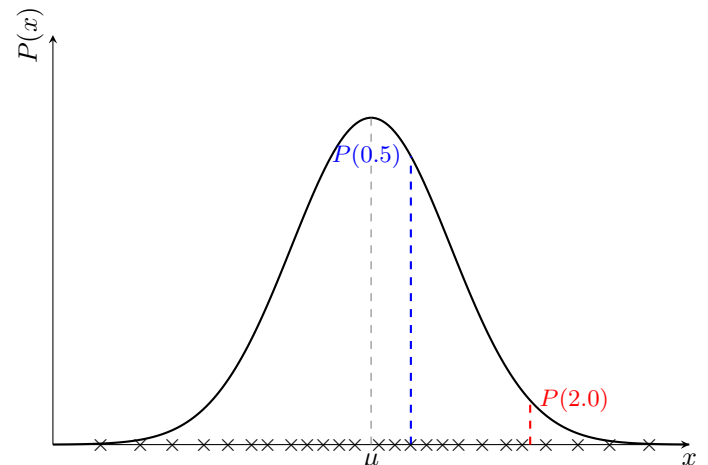
Note: Some statisticians use,

$$\frac{1}{(m-1)} \sum_{i=1}^m (x^{(i)} - \mu)^2$$

instead of

$$\frac{1}{m} \sum_{i=1}^m (x^{(i)} - \mu)^2$$

Example:



Note: $P(x)$ intersections closer to the mean, are highly likely, therefore possibly normal. Meanwhile $P(x)$ intersections farther away from the mean, are unlikely, therefore possibly anomalous.

Setup

Training Set: $\{\vec{x}^{(1)}, \vec{x}^{(2)}, \dots, \vec{x}^{(n)}\}$

Each example has $\vec{x}^{(i)}$ has n features

$$\vec{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

Note: Theoretically we require the features to be statistically independent but the algorithm works just fine even if they aren't.

$$p(\vec{x}) = p(x_1; \mu_1, \sigma_1^2) * p(x_2; \mu_2, \sigma_2^2) * \dots * p(x_n; \mu_n, \sigma_n^2)$$

Example:

$$p(x_1 = \text{high temperature}) = \frac{1}{10}$$

$$p(x_2 = \text{high Vibration}) = \frac{1}{20}$$

$$p(x_1, x_2) = p(x_1) * p(x_2) = \frac{1}{200}$$

Therefore,

$$\therefore p(\vec{x}) = \prod_{j=1}^n p(x_j; \mu_j, \sigma_j^2)$$

Anomaly Detection Algorithm

1. Choose n features x_i that you think might be indicative of anomalous examples.
2. Fit parameters $\mu_1, \mu_2, \dots, \mu_n, \sigma_1^2, \dots, \sigma_n^2$

$$\mu_j = \frac{1}{m} \sum_{i=1}^m x_j^{(i)}; \sigma_j^2 = \frac{1}{m} \sum_{i=1}^m (x_j^{(i)} - \mu_j)^2$$

Vectorised versions,

$$\vec{\mu} = \frac{1}{m} \sum_{i=1}^m \vec{x}^{(i)}; \vec{\mu} = \begin{bmatrix} \mu_1 \\ \mu_2 \\ \vdots \\ \mu_n \end{bmatrix}$$

3. Given a new example n , compute $p(x)$:

$$p(x) = \prod_{j=1}^n p(x_j; \mu_j, \sigma_j^2) = \prod_{j=1}^n \frac{1}{\sqrt{2\pi} \cdot \sigma_j^2} \exp\left(-\frac{(x_j - \mu_j)^2}{2\sigma_j^2}\right)$$

Anomaly if $p(x) < \epsilon$

Algorithm 3 Anomaly Detection

Require: Training set $\{x^{(1)}, x^{(2)}, \dots, x^{(m)}\}$, threshold ϵ , new example x

Ensure: Anomaly decision for x

1: **Step 1: Feature Selection**

2: Choose n features $x_1, x_2, \dots, x_n$ indicative of anomalous behaviour.

3: **Step 2: Fit Gaussian Parameters**

4: **for** $j = 1$ **to** n **do**

5: $\mu_j \leftarrow \frac{1}{m} \sum_{i=1}^m x_j^{(i)}$

6: $\sigma_j^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_j^{(i)} - \mu_j)^2$

7: **end for**

8: *Vectorised:* $\vec{\mu} \leftarrow \frac{1}{m} \sum_{i=1}^m \vec{x}^{(i)}, \quad \vec{\mu} = \begin{bmatrix} \mu_1 \\ \mu_2 \\ \vdots \\ \mu_n \end{bmatrix}$

9: **Step 3: Compute Density for New Example**

10: $p(x) \leftarrow \prod_{j=1}^n \frac{1}{\sqrt{2\pi} \sigma_j} \exp\left(-\frac{(x_j - \mu_j)^2}{2\sigma_j^2}\right)$

11: **Step 4: Classify**

12: **if** $p(x) < \epsilon$ **then**

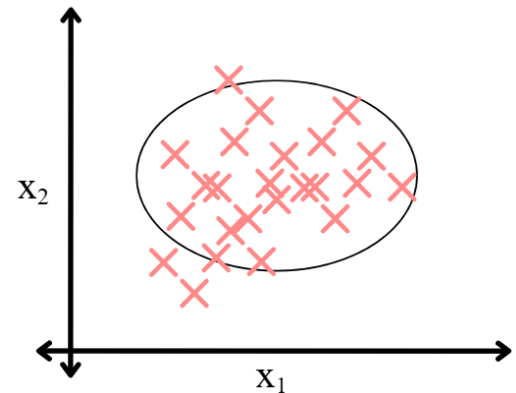
13: **return** *Anomaly*

14: **else**

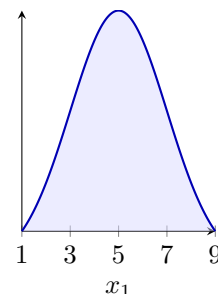
15: **return** *Normal*

16: **end if**

Example: Graphical Visualisation

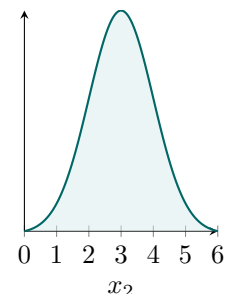


Where the features are distributed as such,



$$\mu_1 = 5, \sigma_1 = 2$$

$$p(x_1; \mu_1, \sigma_1^2)$$



$$\mu_2 = 3, \sigma_2 = 1$$

$$p(x_2; \mu_2, \sigma_2^2)$$

Note: If we have very few labelled anomalous examples it might be better to skip test data and instead have only the training set and Cross-validation set.

Algorithm Evaluation

Fit model $p(x)$ on training set $x^{(1)}, x^{(2)}, \dots, x^{(m)}$

On a cross-validation/test example x , predict

$$y = \begin{cases} 1 & \text{if } p(x) < \epsilon \text{ (anomaly)} \\ 0 & \text{if } p(x) \geq \epsilon \text{ (normal)} \end{cases}$$

Possible evaluation metrics:

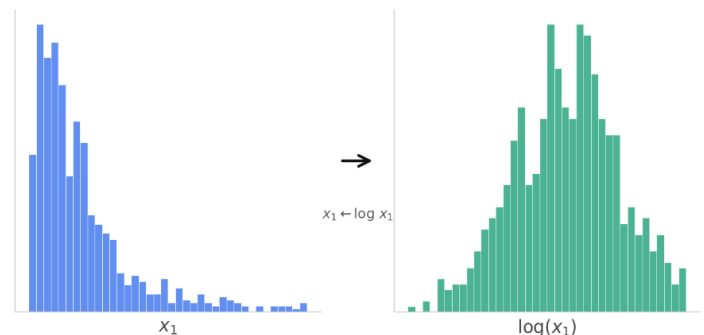
- TP, FP, FN, TN } $\rightarrow$ Confusion Matrix
- precision/recall
- F1-score

Anomaly Detection vs Supervised Learning

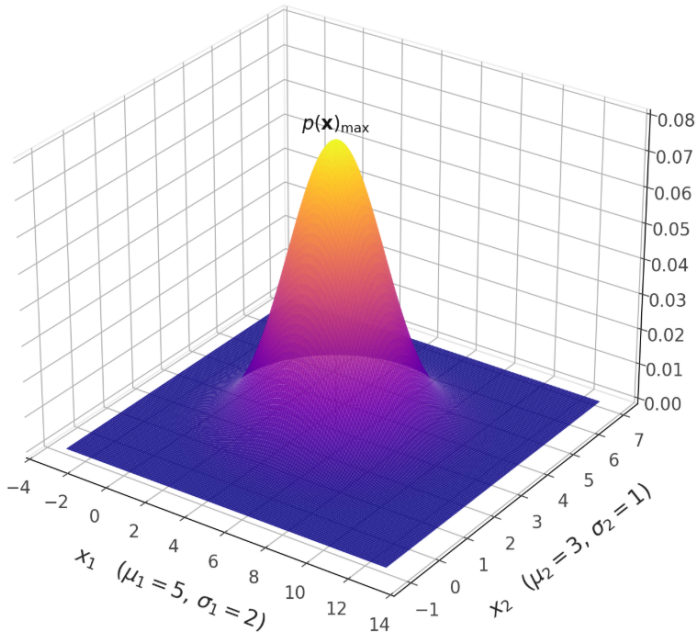
Anomaly Detection	Supervised Learning
Very small number of positive examples ($y = 1$), typically 0–20. Large number of negative examples ($y = 0$).	Large number of both positive and negative examples.
Many <i>different</i> types of anomalies; hard for any algorithm to learn what a future anomaly may look like.	Enough positive examples for the algorithm to learn what they look like.
Future anomalies may look nothing like any seen so far.	Future positive examples likely similar to those in the training set.
fraud, manufacturing, monitoring	spam, image classification, Diseases

Choosing what features to use

- Try to use features which more or less have Gaussian Distribution (plot them and hope for a near a normal distribution).
- If a feature isn't Gaussian we can try to make it more Gaussian.



Note: We can use mathematical transformations to get Gaussian features, common candidates are functions such as $\log(x+c)$, $\sqrt[3]{x}$, $\sqrt{x}$ are some of them.



Note: Values that are more likely will occur closer to the apex, and conversely the lesser likely ones will occur closer to the edges (indicative of anomaly).

Evaluating Algorithm

Assume we have same labelled data, of anomalous ($y = 1$) and non-anomalous ($y = 0$) examples.

Training set:

$\{x^{(1)}, x^{(2)}, \dots, x^{(m)}\}$ (assume normal examples/non-anomalous)

$$y = 0 \quad \forall \quad \text{training examples}$$

Cross Validation set:

$$\{(x_{cv}^{(1)}, y_{cv}^{(1)}), \dots, (x_{cv}^{(m_{cv})}, y_{cv}^{(m_{cv})})\}$$

Test set:

$$\{(x_{test}^{(1)}, y_{test}^{(1)}), \dots, (x_{test}^{(m_{test})}, y_{test}^{(m_{test})})\}$$

Note: Include a few anomalous examples ($y = 1$) but mostly normal examples $y = 0$

Example:

- 10,000 good (normal) engines
- 20 flawed engines (anomalous): generally 2 to 50 anomalous examples

Split:

- **Training Set:** 6000 good engines $\rightarrow$ train algorithm on training set.
- **Cross Validation Set:** 200 good engines ($y = 0$), 10 anomalous ($y = 1$). This is also used to tune ϵ (higher or lower), also tune x_j .
- **Test Set:** 2000 good engines ($y = 0$), 10 anomalous ($y = 1$)

Error Analysis for Anomaly Detection

We want

$p(x) \geq$ Large for normal examples x

$p(x) <$ small for anomalous examples x

Most Common Problem,

$p(x)$ is comparable for normal and anomalous examples

Example: Monitoring Computer Traffic in a data centre

Choose features that might take on unreasonable large or small values in the event of an anomaly.

- x_1 = Memory of use computer
- x_2 = Number of disk access(es)/second
- x_3 = CPU Load
- x_4 = Network Traffic

But, CPU Load may be high while network traffic is low despite both begin within normal range. But this isn't okay and needs to be flagged. We can make our own features.:

$$x_5 = \frac{\text{CPU Load}}{\text{network traffic}}, \quad x_6 = \frac{(\text{CPU Load})^2}{\text{network traffic}}$$

Note: We have to decide which feature to use based on $P(x)$. Needs to be large for normal examples, becomes small for anomaly in the Cross-Validation Set

Recommender Systems

Examples: Predicting Movie Ratings

User rates movies using 0 to 5.

Movie	Alice (1)	Bob (2)	Carol (3)	Dave (4)	x_1 Rom.	x_2 Act.
Destiny	5	5	0	0	0.40	0.00
Superman	5	?	?	0	1.00	0.01
IT	?	4	0	?	0.99	0.00
2024	0	0	5	4	0.10	1.00
Batman	0	0	5	?	0.00	0.00

- $n_u = 4$: number of users
- $n_m = 5$: number of movies
- $\gamma(i, j) = 1$: if user j has rated movie i
 - $\gamma(1, 1) = 1$
 - $\gamma(3, 1) = 0$
- $y^{(i, j)}$ = rating given by user j to movie i
 - defined if and only if $\gamma(i, j) = 1$

For user-1 (Alice): Predict rating for movie i : $w^{(1)} \cdot x^{(i)} + b^{(1)}$

$$w^{[1]} = \begin{bmatrix} 5 \\ 0 \end{bmatrix}, b^{[1]} = 0, x^{[3]} = \begin{bmatrix} 0.99 \\ 0 \end{bmatrix}$$

$$w^{(1)} \cdot x^{(3)} + b^{(1)} = 4.95$$

Cost Function

- For user- j : Predict user j 's rating for movie i as $w^{(j)} \cdot x^{(i)} + b^{(j)}$.
- $m^{(j)}$ = number of movies rated by user j .
- To learn $w^{(j)}, b^{(j)}$

$$\min_{w^{(j)}, b^{(j)}} J(w^{(j)}, b^{(j)}) = \frac{1}{2m^{(j)}} \sum_{i: \gamma(i, j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i, j)})^2 + \frac{\lambda}{2m^{(j)}} \sum_{k=1}^n (w_k^{(j)})^2$$

Note: " $i: \gamma(i, j) = 1$ " refers to i 's where $\gamma(i, j) = 1$ i.e., movies that have been rated, since for others we can't have $y^{(i, j)}$.

Note: we have crossed $m^{(j)}$ as it will be a constant.

i.e., To learn parameters $w^{(j)}, b^{(j)}$ for user j :

$$J(w^{(j)}, b^{(j)}) = \frac{1}{2} \sum_{i: \gamma(i, j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i, j)})^2 + \frac{\lambda}{2} \sum_{k=1}^n (w_k^{(j)})^2$$

To learn parameters $w^{(1)}, b^{(1)}, w^{(2)}, b^{(2)}, \dots, w^{(n_u)}, b^{(n_u)} \forall$ users:

$$J \begin{pmatrix} w^{(1)}, \dots, w^{(n)} \\ b^{(1)}, \dots, b^{(n)} \end{pmatrix} = \frac{1}{2} \sum_{j=1}^{n_u} \sum_{i: \gamma(i, j)=1} \underbrace{(w^{(j)} \cdot x^{(i)} + b^{(j)})}_{f(x)} - y^{(i, j)} \Big)^2 + \frac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^n (w_k^{(j)})^2$$

Collaborative filtering

What if we didn't have features to begin with ?

Examples: Predicting Movie Ratings

User rates movies using 0 to 5.

Movie	Alice (1)	Bob (2)	Carol (3)	Dave (4)	?
Destiny	5	5	0	0	-
Superman	5	?	?	0	-
IT	?	4	0	?	-
2024	0	0	5	4	-
Batman	0	0	5	?	-

Say, we have already learned,

$$w^{(1)} = \begin{bmatrix} 5 \\ 0 \end{bmatrix}, w^{(2)} = \begin{bmatrix} 5 \\ 0 \end{bmatrix}, w^{(3)} = \begin{bmatrix} 0 \\ 5 \end{bmatrix}, w^{(4)} = \begin{bmatrix} 0 \\ 5 \end{bmatrix}$$
$$b^{(1)} = 0, b^{(2)} = 0, b^{(3)} = 0, b^{(4)} = 0$$

using $w^{(j)}.x^{(i)} + b^{(j)}$

$$x^{[1]} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \leftarrow \begin{cases} w^{(1)}.x^{(1)} \approx 5 \\ w^{(2)}.x^{(1)} \approx 5 \\ w^{(3)}.x^{(1)} \approx 0 \\ w^{(4)}.x^{(1)} \approx 0 \end{cases}$$

Note: That is, it's possible to come up with features in collaborative filtering if we have enough examples.

Cost Function

Given $w^{(1)}, b^{(1)}, w^{(2)}, b^{(2)}, \dots, w^{(n_u)}, b^{(n_u)}$,

$$J(x^{(i)}) = \frac{1}{2} \sum_{j: \gamma(i,j)=1} (w^{(j)}.x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{k=1}^n (w_k^{(j)})^2$$

To learn $x^{(1)}, x^{(2)}, \dots, x^{(n_m)}$;

$$J(x^{(1)}, x^{(2)}, \dots, x^{(n_m)}) = \frac{1}{2} \sum_{i=1}^{n_u} \sum_{j: \gamma(i,j)=1} (w^{(j)}.x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{j=1}^{n_m} \sum_{k=1}^n (x_k^{(j)})^2$$

In Summary,

Cost function to learn $w^{(1)}, b^{(1)}, \dots, w^{(n_u)}, b^{(n_u)}$,

$$\min_{w^{(1)}, b^{(1)}, \dots, w^{(n_u)}, b^{(n_u)}} \frac{1}{2} \sum_{j=1}^n \sum_{i: \gamma(i,j)=1} (w^{(j)}.x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \underbrace{\frac{\lambda}{2} \sum \sum (w_k^{(j)})^2}$$

Cost function to learn $x^{(1)}, \dots, x^{(n_m)}$:

$$\min_{w^{(1)}, b^{(1)}, \dots, w^{(n_u)}, b^{(n_u)}} \frac{1}{2} \sum_{j=1}^n \sum_{i: \gamma(i,j)=1} (w^{(j)}.x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \underbrace{\frac{\lambda}{2} \sum \sum (w_k^{(j)})^2}$$

Combining these two,

$$\min_{\begin{pmatrix} w^{(1)}, \dots, w^{(n_u)} \\ b^{(1)}, \dots, b^{(n_u)} \\ x^{(1)}, \dots, x^{(n_m)} \end{pmatrix}} J(w, b, x) = \frac{1}{2} \sum_{(i,j): \gamma(i,j)=1} (w^{(j)}.x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^n (w_k^{(j)})^2 + \frac{\lambda}{2} \sum_{i=1}^{n_m} \sum_{k=1}^n (x_k^{(i)})^2$$

Gradient Descent for Collaborative Filtering

for collaborative filtering,

repeat {

$$w_i^{(j)} = w_i^{(j)} - \alpha \frac{\partial}{\partial w_i^{(j)}} J(w, b, x)$$

$$b^{(j)} = b^{(j)} - \alpha \frac{\partial}{\partial b^{(j)}} J(w, b, x)$$

$$x_k^{(i)} = x_k^{(i)} - \alpha \frac{\partial}{\partial x_k^{(i)}} J(w, b, x)$$

} parameters w, b, x

Binary Labels

Example: Ads shown to users

- 1: engaged after being shown item
- 0: did not engage after being shown item
- ?: not yet shown

Movie	Alice (1)	Bob (2)	Carol (3)	Dave (4)
-	5	5	0	0
-	5	?	?	0
-	?	4	0	?
-	0	0	5	4
-	0	0	5	?

from regression to binary classification

previously,

predict $y^{(i,j)}$ as $w^{(j)}.x^{(i)} + b^{(j)}$

For **binary labels**:

predict that the probability of $y^{(i,j)} = 1$ is given by $g(w^{(j)}.x^{(i)} + b^{(j)})$ where $g(z) = \frac{1}{1+e^{-z}}$

Cost Function for Binary Application

previous cost function,

$$\frac{1}{2} \sum_{(i,j): \gamma(i,j)=1} \underbrace{(w^{(j)}.x^{(i)} + b^{(j)} - y^{(i,j)})^2}_{f(x)} + \frac{\lambda}{2} \sum_{i=1}^{n_m} \sum_{k=1}^n (x_k^{(i)})^2 + \frac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^n (w_k^{(j)})^2$$

Loss for binary labels $y^{(i,j)}$: $f_{(w,b,x)}(x) = g(w^{(j)}.x^{(i)} + b^{(j)})$

$$L(f_{(w,b,x)}(x), y^{(i,j)}) = y^{(i,j)} (f_{(w,b,x)}(x)) - (1 - y^{(i,j)}) \log(1 - f_{(w,b,x)}(x))$$

Loss for all examples,

$$J(w, b, x) = \sum_{(i,j): \gamma(i,j)=1} L(\underbrace{f_{(w,b,x)}(x)}_{g(w^{(j)}.x^{(j)}+b^{(j)})}, y^{(i,j)})$$

Mean Normalisation

Example: Previous Dataset

Movie	Alice (1)	Bob (2)	Carol (3)	Dave (4)	Eve (5)
-	5	5	0	0	? 0
-	5	?	?	0	? 0
-	?	4	0	?	? 0
-	0	0	5	4	? 0
-	0	0	5	?	? 0

Note: Let's add a 5th user that hasn't rated any movies, we could just give all 0's as they haven't rated anything. But since, they haven't rated a single thing the term $w_k^{(j)}$ from the minimisation objective will come out to be 0, giving us $w^{(5)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$ and $b^{[5]} = 0$ will be 0. To resolve this, we can normalise,

$$\begin{aligned} \rightarrow \begin{bmatrix} 5 & 5 & 0 & 0 & ? \\ 5 & ? & ? & 0 & ? \\ ? & 4 & 0 & ? & ? \\ 0 & 0 & 5 & 4 & ? \\ 0 & 0 & 5 & 0 & ? \end{bmatrix} &\rightarrow \begin{bmatrix} 5+5+0+0+? \\ 5+??+0+? \\ ?+4+0+?+? \\ 0+0+5+4+? \\ 0+0+5+0+? \end{bmatrix} \\ \rightarrow \begin{bmatrix} 2.5 \\ 2.5 \\ 2 \\ 2.25 \\ 1.25 \end{bmatrix} = \mu &\rightarrow \underbrace{\begin{bmatrix} 2.5 & 2.2 & -2.5 & -2.5 & ? \\ 2.5 & ? & ? & -2.5 & ? \\ ? & 2 & -2 & ? & ? \\ -2.25 & -2.25 & 2.75 & 1.75 & ? \\ -1.25 & -1.25 & 3.75 & -1.25 & ? \end{bmatrix}}_{y^{(i,j)} \rightarrow w^{(j)}, b^{(j)}, x^{(i)}} \end{aligned}$$

For user j, on movie i predict:

$$w^{(j)}.x^{(i)} + b^{(j)} + \mu_i$$

user-5 (Eve):

$$w^{[5]} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, b^{[5]} = 0, \underbrace{w^{(5)}.x^{(1)} + b^{(5)}}_0 + \mu_1 = 2.5$$

Finding Related Items

The features $x^{(i)}$ of item i are quite hard to interpret.

To find other items related to it, find item k with $x^{(k)}$ similar to $x^{(i)}$

i.e., with smallest distance,

$$\sum_{l=1}^n (x_l^{(k)} - x_l^{(i)})^2, \\ ||x^{(k)} - x^{(i)}||^2$$

Limitations of Collaborative Filtering

Cold start problem. How to ?

- Rank new items that few users have rated ?
- Show something reasonable to new users who have rated few items ?

Use side information about items or users:

- **Item:** Genre, movie stars, studio, ...
- **User:** Demographics (age, gender, location), expressed, preferences, ...

Note: In collaborative filtering, we recommend items based on ratings of users who gave similar ratings as you whereas, in content-based filtering we recommend items to you based on features of user and item to find good match.

Example: Of users and item features

User features

$$x_u^{(i)} \text{ for user } j = \begin{cases} \text{Age} \\ \text{Gender} \\ \text{Country} \\ \text{Movies Matched} \\ \text{Average ratings per genre} \\ \dots \end{cases}$$

Movie features

$$x_m^{(i)} \text{ for movie } i = \begin{cases} \text{Year} \\ \text{Genre} \\ \text{Review} \\ \text{Average ratings per genre} \\ \dots \end{cases}$$

Content-based Filtering: Learning to match

Example:

predicting rating of user j on movie i as,

$$w^{(j)}.x^{(i)} + b^{(j)}$$

where we can use $V_u^{(j)}.V_m^{(i)}$ to calculate $w^{(j)}.x^{(i)}$ computed from $x_u^{(j)}$ and $x_m^{(i)}$

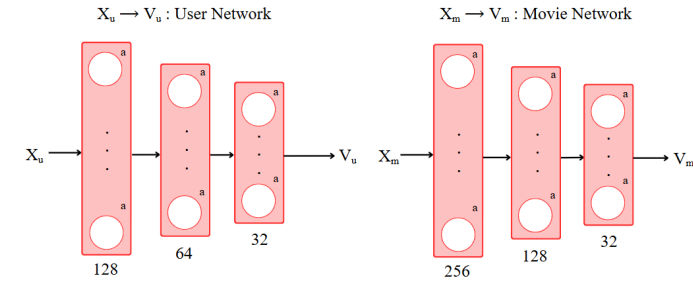
$$\text{User preferences, } V_u = \begin{bmatrix} 4.9 \\ 0.1 \\ \vdots \\ 3.0 \end{bmatrix} \begin{matrix} \text{likes} \\ \text{likes} \\ \vdots \\ \text{likes} \end{matrix}$$

$$\text{movie features, } V_m = \begin{bmatrix} 4.5 \\ 0.2 \\ \vdots \\ 3.5 \end{bmatrix} \begin{matrix} \text{romance} \\ \text{action} \\ \vdots \end{matrix}$$

Note: The dot product will tell us how much a particular user likes a genre.

Deep Learning for Content Based Learning

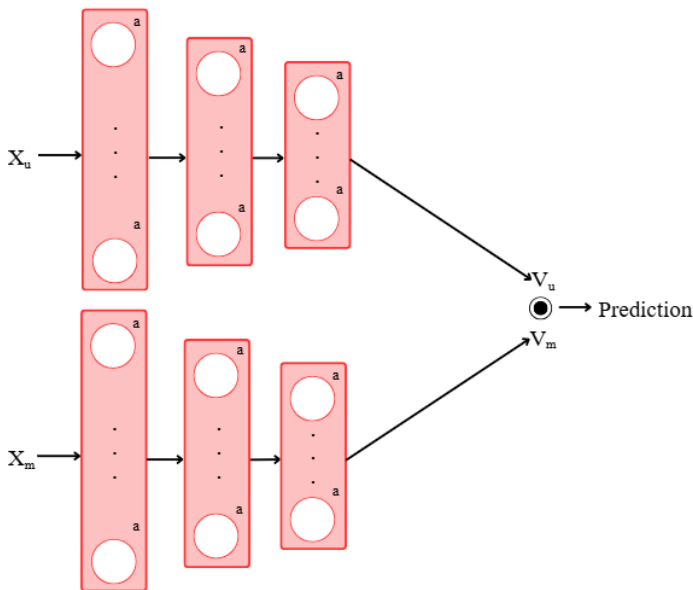
Example: Neural Network Architecture,



prediction: $V_u^{(j)} \cdot V_m^{(i)}$

$g(V_u^{(j)} \cdot V_m^{(i)})$ to predict the probability $y^{(i,j)}$ is 1.

Example: Neural Network Architecture,



Cost function,

$$J = \sum_{(i,j): \gamma(i,j)=1} (v_u^{(j)} \cdot v_m^{(i)} - y^{(i,j)})^2 + NN \text{ regularisation term}$$

Learned user and item vectors:

- $v_u^{(j)}$ is a vector of length 3 that describes user j with features $x_u^{(j)}$
- $v_m^{(i)}$ is a vector of length 32 that describes movie i with features $x_m^{(i)}$

To find movies similar to movie i :

$$\|v_m^{(k)} - v_m^{(i)}\|^2$$

small.

Note: This can be precomputed ahead of time, doesn't need to be computed in real-time

Recommending from a Large Catalogue

How to efficiently find recommendation from a large set of items ?

2 steps: Retrieval and Rank

Retrieval:

- generate large bit of plausible item candidates

Example:

- 1) For each of the last 10 movies watched by the user, find 10 most similar movies

$$\|V_m^{(k)} - V_m^{(i)}\|^2$$

- 2) For most viewed 3 genres, find the top 10 movies,

- 3) Top 20 movies in the country.

Note: Combine Retrieved items into list, removing duplicates and items already watched/purchased.

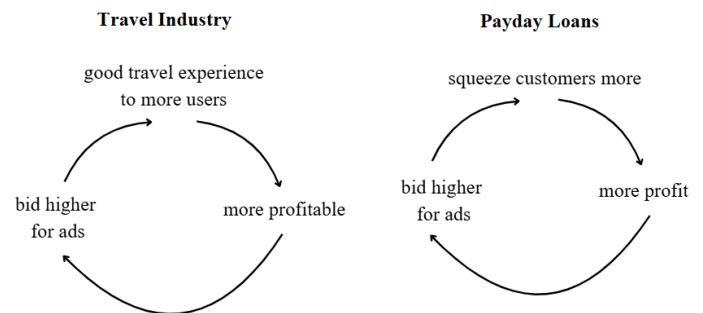
Ranking: Take list retrieval and rank using learned model.

Note: Retrieving more items results in better performance, but slower recommendations.

Note: To analyse/optimize the trade-off, carry out offline experiments to see if retrieving additional more relevant recommendations i.e. $P(y^{(i,j)} = 1)$ of items displayed to user are higher.

Ethical Considerations with Recommender Systems

Example:



- **Amelioration:** Do not accept ads from exploitative businesses.

Note: Maximising user engagement (watch time) has led large social media sites to amplify conspiracy theories and hate/toxicity.

- **Amelioration:** Filter out problematic content such as hate speech, fraud, scams, and violent content.

Note: Can a ranking system that maximises profit rather than user welfare be presented in a transparent way?

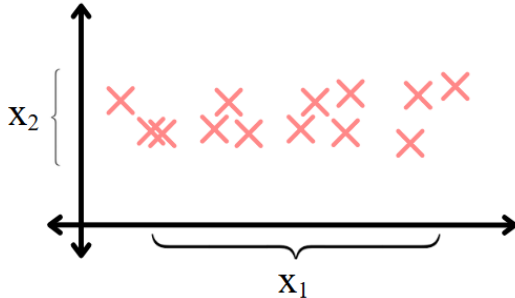
- **Amelioration:** Be transparent with users.

Principal Component Analysis

Example: Car measurements

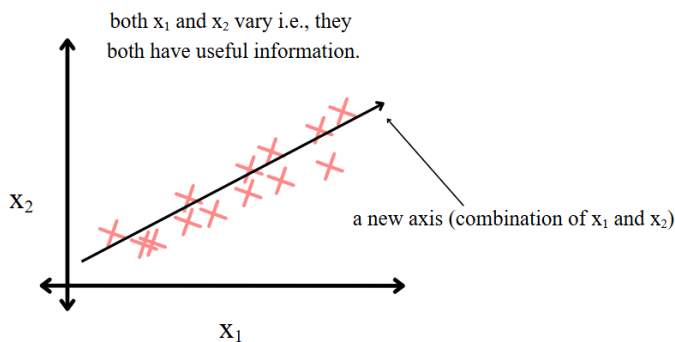
x_1 x_2
length width

Note: Width often not very variable, most cars are a little bit less than the established road width.



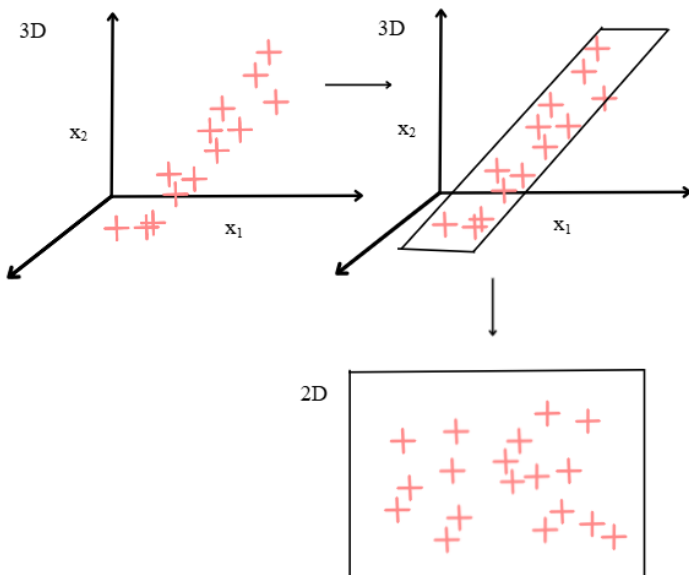
Note: PCA would just take x_1

Example:



Note: "Size" of car can be derived from just 1-axis (only 1-dimensional variance along it), less features to worry about (more simplified).

Example:

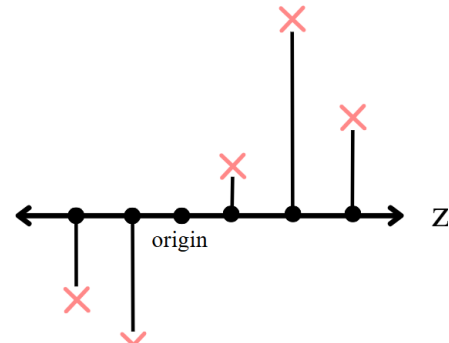


Note: Rotating our 3-d Plot we find that the majority of data lies on a plane. Isolating the points on this plane helps us reduce the dimensions from 3 to 2.

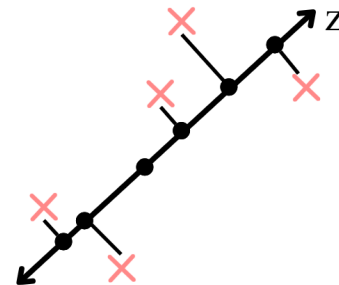
PCA algorithm

Example:

"project" examples onto the axis



Note: an okay choice for z : "captures a lot of the spread"

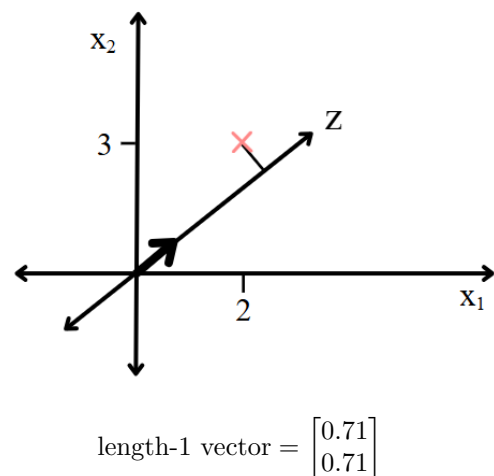


Note: Really good choice: "captures the maximum spread on this axis", with very little perpendicular spread (plausible loss of information)

Note: In PCA terms this is called "Principal Component" (has max variance of data)

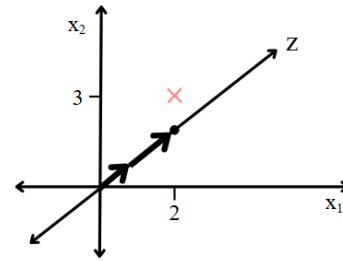
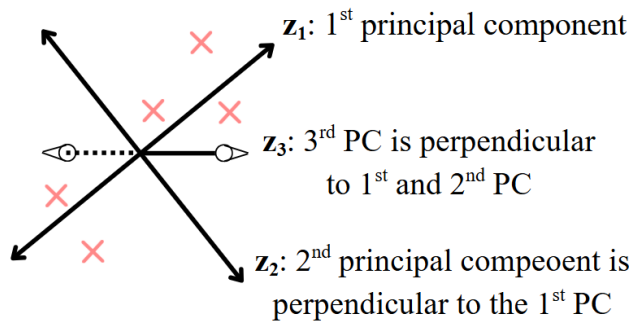
Getting Coordinate on the new axis

Example:



To get projection,

$$\begin{bmatrix} 2 \\ 3 \end{bmatrix} \cdot \begin{bmatrix} 0.71 \\ 0.71 \end{bmatrix} \\ \Rightarrow 2 * 0.71 + 3 * 0.71 = 3.55$$



$$3.55 \cdot \begin{bmatrix} 0.71 \\ 0.71 \end{bmatrix} = \begin{bmatrix} 2.52 \\ 2.52 \end{bmatrix}$$

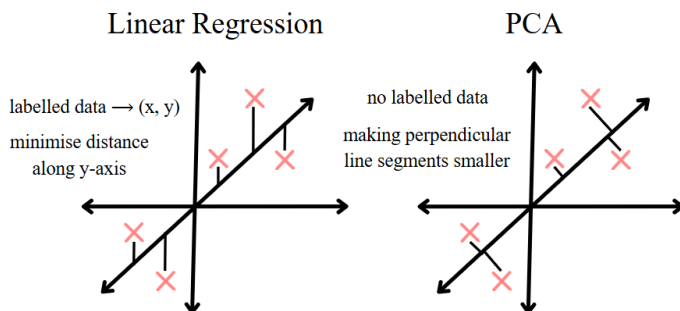
original was $\begin{bmatrix} 2 \\ 3 \end{bmatrix}$ but that information was squashed.

Applications of Principal Component Analysis

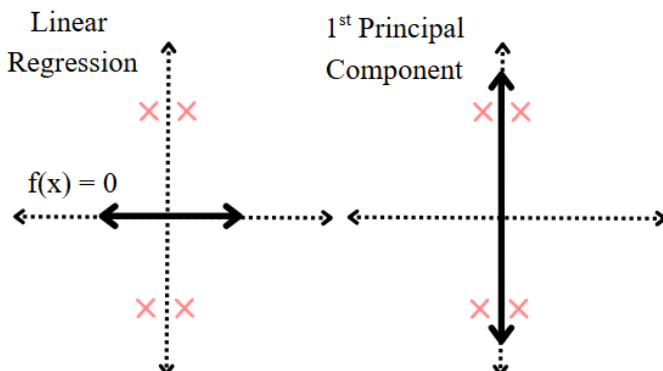
- Visualisations. reduced
- Less frequently used for:
 - Data Compression (to reduce storage/transmission costs)
 - Speeding up training of a supervised learning model

PCA vs Linear Regression

Example:



Example:



Example: we can approximate the original data,

Given $z = 3.55$

We don't have enough information to recover the original (x_1, x_2) [approximately] "reconstruction"

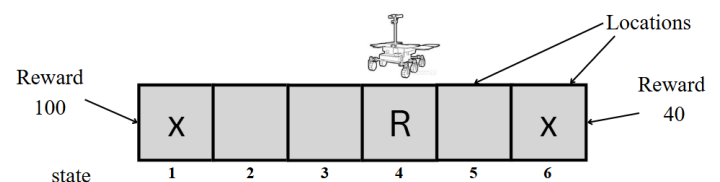
Reinforcement Learning

Example:

position of a helicopter $\rightarrow$ how to move control sticks

State, **S** $\rightarrow$ Action, **A**

Example: Simplified Mars Rover



- Rover can go Left or Right
- State 1 and State 6 are terminal states (rating happens after that)

$$(s, a, R(s), s')$$

where $R(s)$ is reward for being on state (s) , and s' is a new state.

Example:

$$(4, \leftarrow, 0, 3)$$

Example:

$$\text{Return} = 0 + \underbrace{(0.9)}_{\gamma} \cdot 0 + (0.9)^2 \cdot 0 + (0.9)^3 \cdot 100 = 0.729 \times 100 = 72.9$$

where γ is called the Discount Factor.

Generally,

$$\text{Return} = R_1 + \gamma \cdot R_2 + \gamma^2 \cdot R_3 + \dots (\text{until terminal state})$$

Note: As γ^2 , γ^3 and so on 0.9 reduces therefore giving less credit to subsequent steps.

$$\gamma = 0.5$$

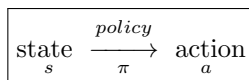
200	50	25	12.5	6.25	40	← return
100	←	←	←	←	←	← reward
1	2	3	4	5	6	

The return depends on the actions you take,

100	2.5	5	10	20	40
100	→	→	→	→	→
1	2	3	4	5	6

Policy

How do we choose actions ?



Example:

100	←	←	→	→	40	seek reward that is nearer
100	←	←	←	←	40	seek larger reward
100	→	→	→	→	40	seek smaller reward
100	←	←	←	→	40	seek bigger reward unless you are one step away from the smaller reward

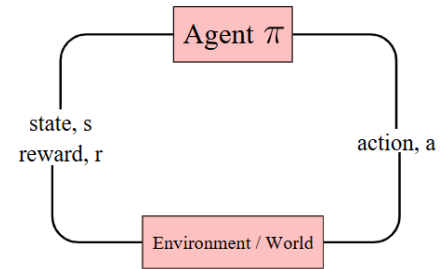
For the last one the policy could be,

$$\begin{cases} \pi(s) = a \\ \pi(s) = \leftarrow \\ \pi(s) = \leftarrow \\ \pi(s) = \leftarrow \\ \pi(s) = \rightarrow \end{cases}$$

Goal of Reinforcement Learning

Find a policy π that tells you what action ($a = \pi(s)$) to take in every state (s) so as to maximise the return.

Markov Decision Processes - MDP



State-Action Value Function

$Q(s, a)$ = Return if you

- start in state, s
- take action a (once)
- then behave optimally after that

Example:

100	←	→				40
100 100	50 125	25 6.25	125 10	6.25 20	40 40	
100	0	0	0	0	40	
1	2	3	4	5	6	

$Q(2, \leftarrow)$ $Q(2, \rightarrow)$

$$Q(2, \rightarrow) = 0 + (0.5) \cdot 0 + (0.5)^2 \cdot 0 + (0.5)^3 (100) = 12.5$$

$$Q(2, \leftarrow) = 0 + (0.5) \cdot 100 = 50$$

Note: "The best possible return from state s is $\max Q(s, a)$."

Applying this on the the example, we get the circular values on the heights. Returns we can get if we land on those states.

We get the *ideal policy*,

100	50	125	25	6.25	12.5	10	6.25	20	40	40
100	0	0	0	0	0	0	0	0	40	40
1	2	3	4	5	6					

← ← → →

ideal policy is,

$$\Pi(s) = \max_a Q(s, a)$$

Bellman Equation

$Q(s, a)$ = Return if you

- start in state s
- take action a (once)
- then behave optimally after that

Variables

- s : current state
- $R(s)$ = reward of current state
- a : current action
- s' : state you get to after taking action a
- a' : action you take in state s'

$$Q(s, a) = R(s) + \gamma \max_a Q(s', a')$$

Example:

$s = 2, a = \rightarrow, s' = 3$

100	100	50	12.5	25	6.25	12.5	10	6.25	20	40	40
100	0	0	0	0	0	0	0	0	0	40	40

$$Q(2, \rightarrow) = R(2) + 0.5 \max_a Q(3, a')$$

$$= 0 + (0.5)25 = 12.5$$

For $s = 4, a = \leftarrow, s' = 3$

$$Q(4, \leftarrow) = R(4) + 0.5 \max_{a'} Q(3, a')$$

$$= 0 + (0.5)25 = 12.5$$

If we are in terminal state, the Bellman equation simplifies to,

$$Q(s, a) = R(s) \because \text{there is no } (s', a')$$

Random (stochastic) environment

Not always do the robot/agent actually achieve the action we say (what if they slip/hit a rock) ?

That is why we compute the average,

$$\text{Expected Return} = \text{Average}(R_1 + \gamma R_2 + \gamma^2 R_3 + \dots)$$

$$= E[R_1 + \gamma R_2 + \gamma^2 R_3 + \dots]$$

This simplifies the Bellman equation, since we can no longer trust s' (the next state might be unexpected),

$$Q(s, a) = R(s) + \gamma \cdot E[\max_{s'} Q(s', a')]$$

Example:

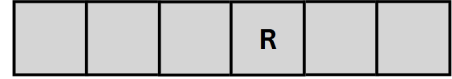
Assume the following policy

	100	$\leftarrow$	$\leftarrow$	$\leftarrow$	$\rightarrow$	40
	1	2	3	4	5	6
should happen	$\leftarrow$	0	0	0	0	100
small deviations	$\leftarrow$	0	0	0	0	0
from policy	$\leftarrow$	0	0	40		

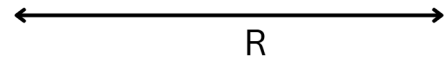
Continuous State Symbols

Example: Discrete v/s Continuous states:

Discrete: possible positions



Continuous: possible positions



Note: Continuous state doesn't have to be just a number it can also be a bunch of numbers that are continuous (state might require multiple numbers to be described)

Example:

$$s = \begin{bmatrix} x \\ y \\ \theta \\ \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix}$$

Example:

Actions:

- do nothing
- left thruster
- right thruster
- main thruster

State:

$$\text{state space} = \begin{bmatrix} x \\ y \\ \dot{x} \\ \dot{y} \\ \dot{\theta} \\ \theta \\ l \\ \gamma \end{bmatrix}$$

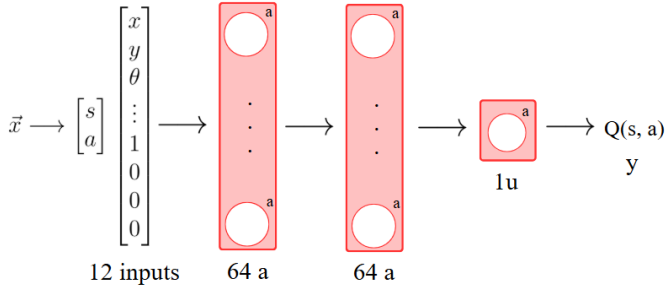
$\rightarrow$ 2D coordinate space (x)
 $\rightarrow$ 2D coordinate space (y)
 $\rightarrow$ horizontal velocity vector
 $\rightarrow$ vertical velocity vector
 $\rightarrow$ angular velocity
 $\rightarrow$ is left leg grounded ?
 $\rightarrow$ is right leg grounded ?

Reward Function:

- getting to landing pad : 100-140
- Additional reward for moving toward/away from pad
- crash : -100
- soft landing : 100
- leg-grounded : 10
- fire main engine : -0.3
- fire side thruster : -0.03

So, we now need to learn a policy, π that given, state, s picks action $a = \pi(s)$ so as to maximise the return. [$\gamma = 0.985$]

Using, Deep Reinforcement Learning



In a state, s , use neural network to compute $Q(s, \text{nothing})$, $Q(s, \text{left})$, $Q(s, \text{main})$, $Q(s, \text{right})$. Pick the action a that maximises $Q(s, a)$.

Using the Bellman,

$$\underbrace{Q(s, a)}_x = R(s) + \underbrace{\gamma \max_{a'} Q(s', a')}_y$$

$$f_{w,B}(x) = y$$

[Take random actions we will observe lots of examples]

Some actions: $(s, a, R(s), s')$

$$10,000 \text{ items } \begin{cases} (s^{(1)}, a^{(1)}, R(s^{(1)}), s'^{(1)}) \\ (s^{(2)}, a^{(2)}, R(s^{(2)}), s'^{(2)}) \end{cases}$$

Using these we can make a dataset,

$$(s^{(1)}, a^{(1)}, R(s^{(1)}), s'^{(1)})$$

where $y^{(1)} = R(s^{(1)}) + \gamma \max_{a'} Q(s'^{(1)}, a')$

$$(s^{(2)}, a^{(2)}, R(s^{(2)}), s'^{(2)})$$

where $y^{(2)} = R(s^{(2)}) + \gamma \max_{a'} Q(s'^{(2)}, a')$

Dataset,

x	y
$x^{(1)} = (s^{(1)}, a^{(1)})$	$y^{(1)}$
$x^{(2)} = (s^{(2)}, a^{(2)})$	$y^{(2)}$
$\vdots$	$\vdots$

• Train the neural network:

– Create a training set of 10,000 examples where:

$$x = (s, a), \quad y = R(s) + \gamma \max_{a'} Q(s', a')$$

– Train Q_{new} such that $Q_{\text{new}}(s, a) \approx y$

– Set $Q \leftarrow Q_{\text{new}}$

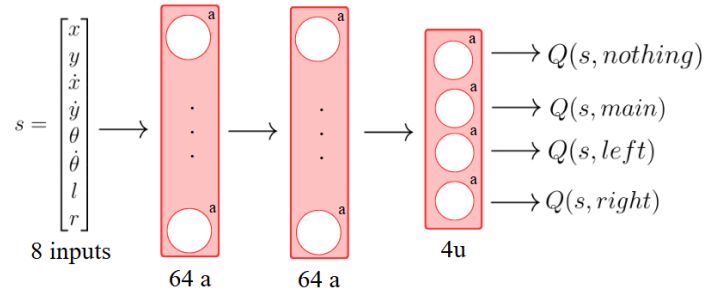
Algorithm 4 Deep Q-Learning with Replay Buffer

```

1: Initialise  $Q(s, a)$  with random weights
2: repeat
3:   Take action  $a$  in environment; observe  $(s, a, R(s), s')$ 
4:   Store  $(s, a, R(s), s')$  in replay buffer  $\mathcal{D}$  ▷ Keep only the
      10,000 most recent tuples
5:   if  $|\mathcal{D}| \geq N$  then
6:     Sample minibatch of  $N = 10,000$  tuples from  $\mathcal{D}$ 
7:     for each  $(s, a, R(s), s')$  in minibatch do
8:       Set  $y = R(s) + \gamma \max_{a'} Q(s', a')$ 
9:     end for
10:    Train  $Q_{\text{new}}$  to minimise  $(Q_{\text{new}}(s, a) - y)^2$ 
11:     $Q \leftarrow Q_{\text{new}}$ 
12:   end if
13: until convergence

```

Example:



In a state s , we input s to the NN, pick the action a that maximises $Q(s, a)$, $R(s) + \gamma \max_{a'} Q(s', a')$

How can we take action while we are still learning

In some states, s ,

– **Option 1:**

– Pick the action a that maximises $Q(s, a)$

– **Option 2:**

– "Exploitation Step": with probability 0.95, pick the action a that maximises $Q(s, a)$

– "Exploration Step": with probability 0.05, pick an action a randomly

Note: Exploration step, so that the model considers new ideas instead of looking away from good ideas or being distracted from them.)

The Learning Algorithm

Example:

Initialise neural network randomly as the initial estimate of $Q(s, a)$.

Repeat:

- Take action in the lunar lander; observe

$$(s, a, R(s), s')$$

- Store the 10,000 most recent $(s, a, R(s), s')$ tuples.
Note: Capping storage prevents memory overflow — this is the *Replay Buffer*.

Note: The setup described above in option is called the ϵ -greedy policy ($\epsilon = 0.05$) [5% Exploring]

Note: A good idea is to start with a high value of ϵ and then to gradually decrease.

Mini-Batch Learning

Breaks a larger dataset, into smaller batches.

Examples: Soft-Update

Set

$$Q \triangleq Q_{new}$$

where Q is composed of w, B and Q_{new} of w_{new}, B_{new}

$$w = 0.01w_{new} + 0.99w$$

$$B = 0.01B_{new} + 0.99B$$

We are only accepting a little bit of the new value not all of it, reducing the odds of throwing everything off.

Limitations of Reinforcement Learning

- Much easier to get to work in a simulation than a real robot/environment !
 - Fewer applications than supervised learning and unsupervised learning !
-